

Verificación de Programas Concurrentes Usando Lógica

Logical Foundations of Concurrent Computation

Jorge A. Pérez
University of Groningen, The Netherlands
j.a.perez [[at]] rug.nl

ECI 2026
(Version of July 30, 2026)

Unrestricted Behavior: Clients and Servers

Outline

Preliminaries

Servers and the ! Modality

- Statics

- Dynamics

- Examples

Clients and Servers in CLL

Process Synchronizations, Informally

- ▶ A 'basic' reduction step:

$$\bar{x}\langle y \rangle.P \mid x(z).Q \longrightarrow P \mid Q\{y/z\}$$

- ▶ In process $\bar{x}(y).P \mid x(z).Q$, the local name y is 'secret' to P (unknown to Q). We have a synchronization:

$$\bar{x}(y).P \mid x(z).Q \longrightarrow (\nu y)(P \mid Q\{y/z\})$$

The scope of the local name y **extends** after reduction!

Process Synchronizations, Informally

- ▶ In the 'basic' reduction step the output $\overline{x}\langle y \rangle$ and the input $x(z)$ are **consumed** after the reduction.

Process Synchronizations, Informally

- ▶ In the 'basic' reduction step the output $\bar{x}\langle y \rangle$ and the input $x(z)$ are **consumed** after the reduction.
- ▶ The **replicated process** $!P$ represents arbitrarily many copies of P :

$$!P \triangleq P \mid P \mid \dots \mid P$$

- ▶ That is, $!P$ can be used zero or many times.
As we will see, $!$ is a logic modality with contraction and weakening.

Process Synchronizations, Informally

- ▶ In the 'basic' reduction step the output $\bar{x}\langle y \rangle$ and the input $x(z)$ are **consumed** after the reduction.
- ▶ The **replicated process** $!P$ represents arbitrarily many copies of P :

$$!P \triangleq P \mid P \mid \dots \mid P$$

- ▶ That is, $!P$ can be used zero or many times.
As we will see, $!$ is a logic modality with contraction and weakening.
- ▶ When we use $!$ for input actions, we obtain a **client-server** behavior:

$$\bar{x}\langle y \rangle.P \mid !x(z).Q \longrightarrow P \mid Q\{y/z\} \mid !x(z).Q$$

Note: the output $\bar{x}\langle y \rangle$ acts as a client request to the server $!x(z).Q$.
The server persists after reduction (for the benefit of other clients).

Process Synchronizations, Informally

- ▶ The **replicated process** $!x(z).P$ represents arbitrarily many copies of $x(z).P$:

$$!x(z).P \triangleq x(z).P \mid x(z).P \mid \cdots \mid x(z).P$$

Process Synchronizations, Informally

- ▶ The **replicated process** $!x(z).P$ represents arbitrarily many copies of $x(z).P$:

$$!x(z).P \triangleq x(z).P \mid x(z).P \mid \cdots \mid x(z).P$$

- ▶ We can use $!$ to obtain a **client-server** behavior:

$$\bar{x}\langle y \rangle.P \mid !x(z).Q \longrightarrow P \mid Q\{y/z\} \mid !x(z).Q$$

Note: the output $\bar{x}\langle y \rangle$ acts as a client request to the server $!x(z).Q$.
The server persists after reduction (for the benefit of other clients).

Process Synchronizations, Informally

- ▶ The **replicated process** $!x(z).P$ represents arbitrarily many copies of $x(z).P$:

$$!x(z).P \triangleq x(z).P \mid x(z).P \mid \dots \mid x(z).P$$

- ▶ We can use $!$ to obtain a **client-server** behavior:

$$\bar{x}\langle y \rangle.P \mid !x(z).Q \longrightarrow P \mid Q\{y/z\} \mid !x(z).Q$$

Note: the output $\bar{x}\langle y \rangle$ acts as a client request to the server $!x(z).Q$.
The server persists after reduction (for the benefit of other clients).

- ▶ When the client sends a **private name**, we then obtain the following:

$$\bar{x}(y).P \mid !x(z).Q \longrightarrow (\nu y)(P \mid Q\{y/z\}) \mid !x(z).Q$$

Process Synchronizations, Informally

- ▶ Client-server interactions can easily lead to **non-terminating behaviors**.
- ▶ **Example:** Consider following the server

$$S \triangleq !x(y).\bar{x}\langle y\rangle.\bar{y}\langle\rangle$$

The server S sends a signal on the channel y sent by the client, and triggers an additional request (to itself). For instance:

$$\begin{aligned} S \mid \bar{x}\langle z\rangle.0 &\longrightarrow S \mid \bar{x}\langle z\rangle.\bar{z}\langle\rangle \\ &\longrightarrow S \mid \bar{z}\langle\rangle \mid \bar{x}\langle z\rangle.\bar{z}\langle\rangle \\ &\longrightarrow S \mid \bar{z}\langle\rangle \mid \bar{z}\langle\rangle \mid \bar{x}\langle z\rangle.\bar{z}\langle\rangle \longrightarrow \dots \end{aligned}$$

- ▶ Type systems can control clients and servers to ensure correctness.

Propositions as Sessions

Linear Logic propositions	$\iff$	Session types describing behavior
Sequent calculus derivations	$\iff$	π -calculus processes
Cut reductions	$\iff$	Communication between processes

Propositions as Sessions

Linear Logic propositions	$\iff$	Session types describing behavior
Sequent calculus derivations	$\iff$	π -calculus processes
Cut reductions	$\iff$	Communication between processes

Today: Incorporating **servers** into the framework.

MAILL: Multiplicative Additive Intuitionistic LL

$A, B ::= 1 \mid A \otimes B \mid A \multimap B \mid A \& B \mid A \oplus B$

$$\begin{array}{c} \overline{A \vdash A} \qquad \overline{\emptyset \vdash 1} \qquad \frac{\otimes R \quad \Gamma_1 \vdash A \quad \Gamma_2 \vdash B}{\Gamma_1, \Gamma_2 \vdash A \otimes B} \qquad \frac{\otimes L \quad \Gamma, A, B \vdash C}{\Gamma, A \otimes B \vdash C} \\[2ex] \frac{\text{Cut} \quad \Gamma \vdash A \quad \Gamma', A \vdash B}{\Gamma, \Gamma' \vdash B} \qquad \frac{\multimap R \quad \Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \qquad \frac{\multimap L \quad \Gamma_1 \vdash A \quad \Gamma_2, B \vdash C}{\Gamma_1, \Gamma_2, A \multimap B \vdash C} \\[2ex] \frac{\& R \quad \Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} \qquad \frac{\& L_i \quad \Gamma, A_i \vdash C}{\Gamma, A_1 \& A_2 \vdash C} \qquad \frac{\oplus R_i \quad \Gamma \vdash A_i}{\Gamma \vdash A_1 \oplus A_2} \qquad \frac{\oplus L \quad \Gamma, A_1 \vdash C \quad \Gamma, A_2 \vdash C}{\Gamma, A_1 \oplus A_2 \vdash C} \end{array}$$

Key Ideas

- We shall consider a language of **processes** (denoted $P, Q, \dots$) that interact by synchronizing on **names** (denoted $x, y, z, \dots$).

Key Ideas

- We shall consider a language of **processes** (denoted $P, Q, \dots$) that interact by synchronizing on **names** (denoted $x, y, z, \dots$).
- Processes can send/receive messages on names, using sequencing and parallel composition. We shall gradually “**extract**” their syntax from proofs.

Key Ideas

- Interpret the logical sequent

$$A_1, \dots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: z : C$$

Key Ideas

- Interpret the logical sequent

$$A_1, \dots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: z : C$$



Process P **offers** protocol C on channel z ...

Key Ideas

- Interpret the logical sequent

$$A_1, \dots, A_n \vdash C$$

as a **typing judgment**, under a suitable reading of propositions as sessions:

$$x_1 : A_1, \dots, x_n : A_n \vdash P :: z : C$$

Process P **offers** protocol C on channel z ...

... by **relying on** protocols $A_1, \dots, A_n$ on channels $x_1, \dots, x_n$.

Outline

Preliminaries

Servers and the ! Modality

- Statics

- Dynamics

- Examples

Clients and Servers in CLL

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for extra €2)
(all main dishes come with a side
of fries)

Desert: Ice Cream, or Cheese
Platter

Drinks: Leffe Tripel (for extra
€3)

$E^{15} \multimap$



Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for extra €2)
(all main dishes come with a side
of fries)

Desert: Ice Cream, or Cheese
Platter

Drinks: Leffe Tripel (for extra
€3)

$$\left(E^{15} \multimap (Sal \& Soup) \otimes \right)$$

Note: We use & to indicate that the kitchen can provide any option you choose.

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for extra €2)
(all main dishes come with a side
of fries)

Desert: Ice Cream, or Cheese
Platter

Drinks: Leffe Tripel (for extra
€3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \ \& \ Soup) \otimes \\ ((Schn \ \& \ Tofu \ \& \ (E \otimes E \multimap Fish)) \end{array} \right)$$

Note: We use $\&$ to indicate that the kitchen can provide any option you choose.

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for extra €2)
(all main dishes come with a side
of fries)

Desert: Ice Cream, or Cheese
Platter

Drinks: Leffe Tripel (for extra
€3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \ \& \ Soup) \otimes \\ ((Schn \ \& \ Tofu \ \& \ (E \otimes E \multimap Fish)) \\ \otimes Fries) \otimes \end{array} \right)$$

Note: We use $\&$ to indicate that the kitchen can provide any option you choose.

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for extra €2)
(all main dishes come with a side
of fries)

Desert: Ice Cream, or Cheese
Platter

Drinks: Leffe Tripel (for extra
€3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \ \& \ Soup) \otimes \\ ((Schn \ \& \ Tofu \ \& \ (E \otimes E \multimap Fish)) \\ \otimes \ Fries) \otimes \\ (IceCr \ \& \ Cheese) \otimes \end{array} \right)$$

Note: We use $\&$ to indicate that the kitchen can provide any option you choose.

Menu at the Linear Logic Cafe

Summer Menu! €15 p.p.

Starter: Greek Salad, or Soup

Main: Chicken Schnitzel,
Tofu, or Salmon (for extra €2)
(all main dishes come with a side
of fries)

Desert: Ice Cream, or Cheese
Platter

Drinks: Leffe Tripel (for extra
€3)

$$E^{15} \multimap \left(\begin{array}{l} (Sal \ \& \ Soup) \otimes \\ ((Schn \ \& \ Tofu \ \& \ (E \otimes E \multimap Fish)) \\ \otimes \ Fries) \otimes \\ (IceCr \ \& \ Cheese) \otimes \\ (E^3 \multimap Beer) \end{array} \right)$$

Note: We use $\&$ to indicate that the kitchen can provide any option you choose.

Restricted vs Unrestricted resources

Intuitionistic (and classical) logic:

$$A \wedge (A \rightarrow B) \vdash B \wedge A \wedge (A \rightarrow B)$$

Linear Logic:

$$A \otimes (A \multimap B) \vdash B$$

$$A \otimes (A \multimap B) \vdash A \otimes (A \multimap B)$$

Restricted vs Unrestricted resources

Intuitionistic (and classical) logic:

$$A \wedge (A \rightarrow B) \vdash B \wedge A \wedge (A \rightarrow B)$$

Linear Logic:

$$A \otimes (A \multimap B) \vdash B$$

$$A \otimes (A \multimap B) \vdash A \otimes (A \multimap B)$$

$$A \otimes (A \multimap B) \vdash B \& (A \otimes (A \multimap B))$$

Unrestricted Resources

- ▶ So far we have only talked about **linear** (restricted) resources.

Unrestricted Resources

- ▶ So far we have only talked about **linear** (restricted) resources.
- ▶ A session type A describes a finite, linear session.

Unrestricted Resources

- ▶ So far we have only talked about **linear** (restricted) resources.
- ▶ A session type A describes a finite, linear session.
- ▶ Composition on disjoint sessions:

$$\frac{\Delta_1 \vdash P :: x : A \quad x : A, \Delta_2 \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

Unrestricted Resources

- ▶ So far we have only talked about **linear** (restricted) resources.
- ▶ A session type A describes a finite, linear session.
- ▶ Composition on disjoint sessions:

$$\frac{\Delta_1 \vdash P :: x : A \quad x : A, \Delta_2 \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$



Divide all the resources between P and Q

Unrestricted Resources

- ▶ So far we have only talked about **linear** (restricted) resources.
- ▶ A session type A describes a finite, linear session.
- ▶ Composition on disjoint sessions:

$$\frac{\Delta_1 \vdash P :: x : A \quad x : A, \Delta_2 \vdash Q :: z : C}{\Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

- ▶ Question: How to introduce **unrestricted** (i.e. non-linear) sessions?

The ! Modality

$!A$ ‘of course A ’, or ‘**bang** A ’, or ‘**exponential** A ’

Some principles:

$$!A \vdash A$$

$$!A \vdash !A \otimes !A$$

$$!A \vdash 1$$

$$\frac{A \vdash B}{!A \vdash !B}$$

► $!A$ satisfies contraction and weakening.

The ! Modality

$!A$ ‘of course A ’, or ‘**bang** A ’, or ‘**exponential** A ’

Some principles:

$$!A \vdash A$$

$$!A \vdash !A \otimes !A$$

$$!A \vdash 1$$

$$\frac{A \vdash B}{!A \vdash !B}$$

- ▶ $!A$ satisfies contraction and weakening.
- ▶ A list-like presentation: $!A \vdash 1 \ \& \ (A \otimes !A)$

The ! Modality

$!A$ ‘of course A ’, or ‘**bang** A ’, or ‘**exponential** A ’

Some principles:

$$!A \vdash A$$

$$!A \vdash !A \otimes !A$$

$$!A \vdash 1$$

$$\frac{A \vdash B}{!A \vdash !B}$$

- ▶ $!A$ satisfies contraction and weakening.
- ▶ A list-like presentation: $!A \vdash 1 \ \& \ (A \otimes !A)$
- ▶ $A \otimes !(A \multimap B) \vdash B \otimes !(A \multimap B)$

The ! Modality

Some useful properties of !:

► $!(A \& B) \dashv\vdash !A \otimes !B$

The ! Modality

Some useful properties of !:

▶ $!(A \& B) \dashv\vdash !A \otimes !B$

▶ $!(A \otimes B) \dashv\vdash !A \otimes !B$

The ! Modality

Some useful properties of !:

- ▶ $!(A \& B) \not\vdash !A \otimes !B$
- ▶ $!(A \otimes B) \not\vdash !A \otimes !B$
- ▶ $!(A \& B) \vdash !A \& !B$
- ▶ $!A \& !B \not\vdash !(A \& B)$

The ! Modality

Some useful properties of !:

- ▶ $!(A \& B) \dashv\vdash !A \otimes !B$
- ▶ $!(A \otimes B) \dashv\vdash !A \otimes !B$
- ▶ $!(A \& B) \vdash !A \& !B$
- ▶ $!A \& !B \not\vdash !(A \& B)$
- ▶ $!A \dashv\vdash !!A$

The ! Modality as a Session Type

- ▶ $!A$: Unlimited copies of the session A . That is, a server for session A

The ! Modality as a Session Type

- ▶ $!A$: Unlimited copies of the session A . That is, a server for session A
- ▶ We refine our judgment with two kinds of contexts:

$$\Gamma; \Delta \vdash C$$

Unrestricted resources appear in Γ , restricted resources appear in Δ .

The ! Modality as a Session Type

- ▶ $!A$: Unlimited copies of the session A . That is, a server for session A
- ▶ We refine our judgment with two kinds of contexts:

$$\Gamma; \Delta \vdash C$$

Unrestricted resources appear in Γ , restricted resources appear in Δ .

- ▶ This variant is known as DILL: Dual Intuitionistic Linear Logic.

The ! Modality as a Session Type

- ▶ $!A$: Unlimited copies of the session A . That is, a server for session A
- ▶ We refine our judgment with two kinds of contexts:

$$\Gamma; \Delta \vdash C$$

Unrestricted resources appear in Γ , restricted resources appear in Δ .

- ▶ This variant is known as DILL: Dual Intuitionistic Linear Logic.
- ▶ The judgment $A_1, \dots, A_k; B_1, \dots, B_n \vdash C$ stands for

$$!A_1 \otimes \dots \otimes !A_k \otimes B_1 \otimes \dots \otimes B_n \multimap C$$

The ! Modality as a Session Type

- ▶ $!A$: Unlimited copies of the session A . That is, a server for session A
- ▶ We refine our judgment with two kinds of contexts:

$$\Gamma; \Delta \vdash C$$

Unrestricted resources appear in Γ , restricted resources appear in Δ .

- ▶ This variant is known as DILL: Dual Intuitionistic Linear Logic.
- ▶ The judgment $A_1, \dots, A_k; B_1, \dots, B_n \vdash C$ stands for

$$!A_1 \otimes \dots \otimes !A_k \otimes B_1 \otimes \dots \otimes B_n \multimap C$$

- ▶ Unrestricted resources are non-linear and can be shared. The cut rule becomes:

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

The ! Modality as a Session Type

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

$$\frac{\Gamma; \Delta_1 \vdash P :: y : A \quad \Gamma; \Delta_2 \vdash Q :: x : B}{\Gamma; \Delta_1, \Delta_2 \vdash \bar{x}\langle y \rangle.(P \mid Q) :: x : A \otimes B}$$

$$\frac{}{\Gamma; x : A \vdash [y \leftarrow x] :: y : A}$$

...etc

The ! Modality as a Session Type

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \quad \Gamma; x : A, \Delta_2 \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P \mid Q) :: z : C}$$

$$\frac{\Gamma; \Delta_1 \vdash P :: y : A \quad \Gamma; \Delta_2 \vdash Q :: x : B}{\Gamma; \Delta_1, \Delta_2 \vdash \bar{x}\langle y \rangle.(P \mid Q) :: x : A \otimes B}$$

$$\frac{}{\Gamma; x : A \vdash [y \leftarrow x] :: y : A}$$

...etc

Plus contraction and weakening for unrestricted resources (in Γ):

$$\frac{x : A, y : A, \Gamma; \Delta \vdash P :: z : C}{x : A, \Gamma; \Delta \vdash P\{x/y\} :: z : C}$$

$$\frac{\Gamma; \Delta \vdash P :: z : C}{x : A, \Gamma; \Delta \vdash P :: z : C}$$

The Concurrent Interpretation, Now With Servers

To extend the interpretation, we proceed as before:

- ▶ We consider right and left rules for the modality $!A$.
- ▶ We annotate the rules with suitable process constructs, expressing servers and clients.
- ▶ We identify principles that enable principal cut reductions, which can be interpreted as process reductions.

The Concurrent Interpretation, Now With Servers

To extend the interpretation, we proceed as before:

- ▶ We consider right and left rules for the modality $!A$.
- ▶ We annotate the rules with suitable process constructs, expressing servers and clients.
- ▶ We identify principles that enable principal cut reductions, which can be interpreted as process reductions.

In the presence of unrestricted resources, we have an implicit source of client-servers behaviors, given by Γ .

We can also be explicit via the modality $!A$ at the level of propositions.

The Concurrent Interpretation, Now With Servers

!-move

$$\frac{u : A, \Gamma; y : A, \Delta \vdash Q :: z : C}{u : A, \Gamma; \Delta \vdash (\nu y) \bar{u}(y).Q :: z : C}$$

!-cut

$$\frac{\Gamma; \emptyset \vdash P :: y : A \quad \Gamma, x : A; \Delta \vdash Q :: z : C}{\Gamma; \Delta \vdash (\nu x)(!x(y).P \mid Q) :: z : C}$$

!R

$$\frac{\Gamma; \emptyset \vdash P :: y : A}{\Gamma; \emptyset \vdash !u(y).P :: u : !A}$$

!L

$$\frac{u : A, \Gamma; \Delta \vdash P :: z : C}{\Gamma; \Delta, x : !A \vdash P\{x/u\} :: z : C}$$

With the following reduction rule:

$$(\nu u)(!u(y).P \mid (\nu y)(\bar{u}(y).Q)) \longrightarrow (\nu u)(!u(y).P \mid (\nu y)(P \mid Q))$$

The ! Modality: Servers

The right rule for !:

$$\frac{\Gamma; \emptyset \vdash \quad A}{\Gamma; \emptyset \vdash !A}$$

- ▶ We can derive $!A$ from A if the proof of A (i.e., the process P) does **not** depend on linear/restricted resources.

The ! Modality: Servers

The right rule for !:

$$\frac{\Gamma; \emptyset \vdash P :: y : A}{\Gamma; \emptyset \vdash !u(y).P :: u : !A}$$

- ▶ We can derive $!A$ from A if the proof of A (i.e., the process P) does **not** depend on linear/restricted resources.
- ▶ The server $!u(y).P$ is always ready to receive a request (with a name) on u , and to provide P for that request.

The ! Modality: Clients

$$\frac{u : A, \Gamma; \Delta \vdash \quad C}{\Gamma; x : !A, \Delta \vdash \quad C}$$

$$\frac{u : A, \Gamma; y : A, \Delta \vdash \quad C}{u : A, \Gamma; \Delta \vdash \quad C}$$

The ! Modality: Clients

$$\frac{u : A, \Gamma; \Delta \vdash P :: z : C}{\Gamma; x : !A, \Delta \vdash P\{x/u\} :: z : C}$$

$$\frac{u : A, \Gamma; y : A, \Delta \vdash C}{u : A, \Gamma; \Delta \vdash C}$$

- We can move $!A$ from the restricted to the unrestricted section.

The ! Modality: Clients

$$\frac{u : A, \Gamma; \Delta \vdash P :: z : C}{\Gamma; x : !A, \Delta \vdash P\{x/u\} :: z : C}$$

$$\frac{u : A, \Gamma; y : A, \Delta \vdash Q :: z : C}{u : A, \Gamma; \Delta \vdash (\nu y) \bar{u}\langle y \rangle. Q :: z : C}$$

- ▶ We can move $!A$ from the restricted to the unrestricted section.
- ▶ A client can request a copy of A by creating a new name and sending the request to $!A$.

Client-Server Interaction

We expect to obtain the following reduction rule:

$$(\nu u)(!u(y).P \mid (\nu y)(\bar{u}\langle y \rangle \mid Q)) \longrightarrow (\nu u)(!u(y).P \mid (\nu y)(P \mid Q))$$

Client-Server Interaction

We expect to obtain the following reduction rule:

$$(\nu u)(!u(y).P \mid (\nu y)(\bar{u}\langle y \rangle \mid Q)) \longrightarrow (\nu u)(!u(y).P \mid (\nu y)(P \mid Q))$$

For convenience, we will also use the **derived composition rule** for servers:

$$\frac{\begin{array}{c} \text{!-cut} \\ \Gamma; \emptyset \vdash P :: y : A \quad \Gamma, x : A; \Delta \vdash Q :: z : C \end{array}}{\Gamma; \Delta \vdash (\nu x)(!x(y).P \mid Q) :: z : C}$$

The ! Modality: Client-Server Interaction (1)

The interaction between 'move' and '!-cut' is simplified using a '!-cut' and 'cut':

$$\frac{\Gamma; \emptyset \vdash P :: x : A \quad \frac{\Gamma, u : A; \Delta, y : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu y) \bar{u} \langle y \rangle . Q :: z : C}}{\Gamma; \Delta \vdash (\nu u) (!u(x).P \mid (\nu y) \bar{u} \langle y \rangle . Q) :: z : C}$$

$\longrightarrow$

The ! Modality: Client-Server Interaction (1)

The interaction between 'move' and '!-cut' is simplified using a '!-cut' and 'cut':

$$\frac{\frac{\Gamma; \emptyset \vdash P :: x : A \quad \frac{\Gamma, u : A; \Delta, y : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu y) \bar{u} \langle y \rangle . Q :: z : C}}{\Gamma; \Delta \vdash (\nu u) (!u(x).P \mid (\nu y) \bar{u} \langle y \rangle . Q) :: z : C}}{\longrightarrow} \frac{\Gamma; \emptyset \vdash P :: x : A \quad \Gamma, u : A; \Delta, y : A \vdash Q :: z : C}{\quad}$$

The ! Modality: Client-Server Interaction (1)

The interaction between 'move' and '!-cut' is simplified using a '!-cut' and 'cut':

$$\frac{\Gamma; \emptyset \vdash P :: x : A \quad \frac{\Gamma, u : A; \Delta, y : A \vdash Q :: z : C}{\Gamma, u : A; \Delta \vdash (\nu y) \bar{u} \langle y \rangle . Q :: z : C}}{\Gamma; \Delta \vdash (\nu u) (!u(x).P \mid (\nu y) \bar{u} \langle y \rangle . Q) :: z : C}$$

$\longrightarrow$

$$\frac{\Gamma; \emptyset \vdash P :: x : A \quad \frac{\Gamma; \emptyset \vdash P :: x : A \quad \Gamma, u : A; \Delta, y : A \vdash Q :: z : C}{\Gamma; \Delta, y : A \vdash (\nu u) (!u(x).P \mid Q) :: z : C}}{\Gamma; \Delta \vdash (\nu u) (P\{y/x\} \mid (\nu u) (!u(x).P \mid Q)) :: z : C}$$

The ! Modality: Client-Server Interaction (2)

Now the interaction between the right/left rules for $!A$. Write T instead of $z : C$:

$$\frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma; \emptyset \vdash !u(x).P :: u : !A} \quad \frac{\frac{\Gamma, u : A; \Delta, y : A \vdash Q :: T}{\Gamma, u : A; \Delta \vdash (\nu y)\bar{u}\langle y \rangle.Q :: T}}{\Gamma; \Delta, u : !A \vdash (\nu y)\bar{u}\langle y \rangle.Q :: T}}{\Gamma; \Delta \vdash (\nu u)(!u(x).P \mid (\nu y)\bar{u}\langle y \rangle.Q) :: T} \longrightarrow$$

The ! Modality: Client-Server Interaction (2)

Now the interaction between the right/left rules for $!A$. Write T instead of $z : C$:

$$\begin{array}{c}
 \frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma; \emptyset \vdash !u(x).P :: u : !A} \quad \frac{\frac{\Gamma, u : A; \Delta, y : A \vdash Q :: T}{\Gamma, u : A; \Delta \vdash (\nu y)\bar{u}\langle y \rangle.Q :: T}}{\Gamma; \Delta, u : !A \vdash (\nu y)\bar{u}\langle y \rangle.Q :: T} \\
 \hline
 \Gamma; \Delta \vdash (\nu u)(!u(x).P \mid (\nu y)\bar{u}\langle y \rangle.Q) :: T \\
 \longrightarrow \\
 \frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma, u : A; \emptyset \vdash P\{y/x\} :: y : A} \quad \Gamma, u : A; \Delta, y : A \vdash Q :: T \\
 \hline
 \Gamma, u : A; \Delta \vdash (\nu y)(P\{y/x\} \mid Q) :: T
 \end{array}$$

The ! Modality: Client-Server Interaction (2)

Now the interaction between the right/left rules for $!A$. Write T instead of $z : C$:

$$\frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma; \emptyset \vdash !u(x).P :: u : !A} \quad \frac{\frac{\Gamma, u : A; \Delta, y : A \vdash Q :: T}{\Gamma, u : A; \Delta \vdash (\nu y)\bar{u}\langle y \rangle.Q :: T}}{\Gamma; \Delta, u : !A \vdash (\nu y)\bar{u}\langle y \rangle.Q :: T}}{\Gamma; \Delta \vdash (\nu u)(!u(x).P \mid (\nu y)\bar{u}\langle y \rangle.Q) :: T}$$

$\longrightarrow$

$$\frac{\frac{\Gamma; \emptyset \vdash P :: x : A}{\Gamma, u : A; \emptyset \vdash P\{y/x\} :: y : A} \quad \Gamma, u : A; \Delta, y : A \vdash Q :: T}{\frac{\Gamma, u : A; \Delta \vdash (\nu y)(P\{y/x\} \mid Q) :: T}{\Gamma; \Delta, u : !A \vdash (\nu x)(P\{y/x\} \mid Q) :: T}}{\Gamma; \Delta \vdash (\nu u)(!u(x).P \mid (\nu y)(P\{y/x\} \mid Q)) :: T}$$

List-like Specification for Servers

$$\frac{u : A; \emptyset \vdash \dots :: w : 1 \& (A \otimes !A)}{\emptyset; u : !A \vdash \dots :: w : 1 \& (A \otimes !A)}$$

List-like Specification for Servers

$$\begin{array}{c}
 \frac{u : A; \emptyset \vdash \dots :: w : 1}{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \dots, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)} \quad \frac{}{u : A; \emptyset \vdash \dots :: w : A \otimes !A} \\
 \hline
 \frac{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \dots, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}{\emptyset; u : !A \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \dots, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}
 \end{array}$$

List-like Specification for Servers

$$\begin{array}{c}
 \frac{u : A; \emptyset \vdash \overline{w}\langle \rangle :: w : 1 \quad \frac{}{u : A; \emptyset \vdash \dots :: w : A \otimes !A}}{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)} \\
 \frac{u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}{\emptyset; u : !A \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)}
 \end{array}$$

List-like Specification for Servers

$$\begin{array}{c}
 u : A; \emptyset \vdash \overline{u}(x).[y \leftarrow x] :: y : A \\
 u : A; \emptyset \vdash [w \leftarrow u] :: w :: !A \\
 \hline
 u : A; \emptyset \vdash \overline{w}\langle \rangle :: w : 1 \quad u : A; \emptyset \vdash \overline{w}(y).(\overline{u}(x).[y \leftarrow x]) \mid [w \leftarrow u] :: w : A \otimes !A \\
 \hline
 u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A) \\
 \hline
 u : A; \emptyset \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A) \\
 \hline
 \emptyset; u : !A \vdash w \triangleright \left\{ \begin{array}{l} \text{inl} : \overline{w}\langle \rangle, \\ \text{inr} : \dots \end{array} \right\} :: w : 1 \& (A \otimes !A)
 \end{array}$$

Bar Tending at the Linear Logic Cafe

$$\begin{array}{c}
 \frac{\frac{\frac{\emptyset; e : E \otimes E \otimes E \vdash \text{depo}(e, x) :: x : 1}{u : Keg; e : E^3 \vdash (\nu x)(\text{depo}(e, x) \mid x().\bar{u}(k).\text{pour}(k, b)) :: b : Beer} \quad \frac{\frac{\frac{\emptyset; k : Keg \vdash \text{pour}(k, b) :: b : Beer}{u : Keg; \emptyset \vdash \bar{u}(k).\text{pour}(k, b) :: b : Beer}}{u : Keg; x : 1 \vdash x().\bar{u}(k).\text{pour}(k, b) :: b : Beer}}{u : Keg; \emptyset \vdash b(e).(\nu x)(\text{depo}(e, x) \mid x().\bar{u}(k).\text{pour}(k, b)) :: b : E^3 \multimap Beer} \\
 u : Keg; \emptyset \vdash !t(b).b(e).(\nu x)(\text{depo}(e, x) \mid x().\bar{u}(k).\text{pour}(k, b)) :: t : !(E^3 \multimap Beer)
 \end{array}$$

Outline

Preliminaries

Servers and the ! Modality

Statics

Dynamics

Examples

Clients and Servers in CLL

Clients and Servers in CLL

We have just seen clients and servers in the intuitionistic setting (DILL).

In Wadler's CP, clients and servers appear differently.

- ▶ We have exponential modalities: $!A$ for servers, and its dual $?A$ for clients. At the level of judgments, there is no separation between linear and unrestricted resources.

Clients and Servers in CLL

- ▶ There is one rule for servers:

$$\frac{P \vdash ?\Gamma, y : A}{!x(y).P \vdash ?\Gamma, x : !A} [!]$$

? Γ ensures: A process may only provide a server if it is implemented by communicating only with other servers.

Clients and Servers in CLL

- ▶ There is one rule for servers:

$$\frac{P \vdash ?\Gamma, y : A}{!x(y).P \vdash ?\Gamma, x : !A} [!]$$

? Γ ensures: A process may only provide a server if it is implemented by communicating only with other servers.

- ▶ There are three rules for clients (one client, zero clients, multiple clients):

$$\frac{Q \vdash \Delta, y : A}{\bar{x}(y).Q \vdash \Delta, x : ?A} [?]$$

$$\frac{Q \vdash \Delta}{Q \vdash \Delta, x : ?A} [W]$$

$$\frac{Q \vdash \Delta, x : ?A, x' : ?A}{Q\{x/x'\} \vdash \Delta, x : ?A} [C]$$

Clients and Servers in CLL

- ▶ There is one rule for servers:

$$\frac{P \vdash ?\Gamma, y : A}{!x(y).P \vdash ?\Gamma, x : !A} [!]$$

? Γ ensures: A process may only provide a server if it is implemented by communicating only with other servers.

- ▶ There are three rules for clients (one client, zero clients, multiple clients):

$$\frac{Q \vdash \Delta, y : A}{\bar{x}(y).Q \vdash \Delta, x : ?A} [?]$$

$$\frac{Q \vdash \Delta}{Q \vdash \Delta, x : ?A} [W]$$

$$\frac{Q \vdash \Delta, x : ?A, x' : ?A}{Q\{x/x'\} \vdash \Delta, x : ?A} [C]$$

- ▶ Accordingly, we have three reductions with a server $!x(y).P$ implementing $!A$:

$$(\nu x)(!x(y).P \mid \bar{x}(y).Q) \longrightarrow (\nu y)(P \mid Q)$$

$$(\nu x)(!x(y).P \mid Q) \longrightarrow Q$$

$$(\nu x)(!x(y).P \mid Q\{x/x'\}) \longrightarrow (\nu x)(x(y).P \mid (\nu x')(x'(y).P \mid Q))$$

Taking Stock

- Concurrent interpretation of LL: statics and dynamics
- Left and right rules per connective - rely and guarantee interactive behaviors
- Cut reductions and process synchronizations
- A look at the correctness properties ensured by the logic-based type system
- The computational interpretation of proof transformations
- Deadlock freedom (DF)
- Asynchronous (classical) processes and DF.
- Clients and servers in DILL (and CLL).